



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N.º 08

NOMBRE COMPLETO: Rosas Cañada Abraham

N.º de Cuenta: 318307866

GRUPO DE LABORATORIO: 06

GRUPO DE TEORÍA: 03

SEMESTRE: 2026-2

FECHA DE ENTREGA LÍMITE: 26/04/2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

Archivos modificados

Para el desarrollo de la práctica se modificaron principalmente los siguientes archivos:

- main.cpp: archivo principal donde se integró toda la lógica de la escena, incluyendo la creación de luces puntuales y spotlights, el control por teclado, la pecera, el pez abisal, la nave y el renderizado de transparencia.
- shaders/shader_light.frag: shader de fragmentos donde se ajustó el tratamiento de normales e iluminación del vidrio para que las paredes de la pecera se iluminaran correctamente hacia el interior.
- shaders/shader_light.vert: archivo asociado al pipeline de iluminación, utilizado junto con el fragment shader para procesar posición, normales y matrices de transformación necesarias para el cálculo de luz.

Razón de modificación: estos archivos concentran la lógica de iluminación, transformaciones jerárquicas, materiales, transparencia y cálculo de dirección/posición de luces dinámicas, por lo que fueron los puntos adecuados para implementar todos los requisitos de la práctica.

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

1. Agregar luz de tipo spotlight para la nave de tal forma que al avanzar (mover con teclado hacia dirección de X negativa) ilumine con un spotlight en el suelo hacia adelante y al retroceder (mover con teclado hacia dirección de X positiva) ilumine con un spotlight el suelo hacia atrás. Son dos spotlights diferentes que se prenderán y apagarán de acuerdo a la dirección en la que esté moviéndose el helicóptero.

Se agregaron dos spotlights distintos ligados a la nave, uno frontal y uno trasero. La lógica consiste en calcular dos posiciones locales sobre el modelo de la nave, transformarlas a coordenadas del mundo mediante la matriz `modelNaveRender`, y activar solo uno de los spotlights según la dirección de movimiento sobre el eje X. Esta solución sigue el enfoque habitual en OpenGL de actualizar posición y dirección de la luz a partir de la transformación del objeto al que está ligada.

```

naveX = 0.0f + mainWindow.getmuevehelicoptero();

glm::mat4 modelNaveRender = glm::mat4(1.0f);
modelNaveRender = glm::translate(modelNaveRender, glm::vec3(naveX, 5.0f, 6.0f));
modelNaveRender = glm::scale(modelNaveRender, glm::vec3(3.0f, 3.0f, 3.0f));
modelNaveRender = glm::rotate(modelNaveRender, -180.0f * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
modelNaveRender = glm::rotate(modelNaveRender, -180.0f * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));

modelNave = modelNaveRender;

naveSpotPosFrontal = glm::vec3(modelNaveRender * glm::vec4(0.7f, 0.0f, 0.0f, 1.0f));
naveSpotPosTrasero = glm::vec3(modelNaveRender * glm::vec4(-0.7f, 0.0f, 0.0f, 1.0f));
naveSpotPosOff = glm::vec3(modelNaveRender * glm::vec4(0.0f, -35.0f, 0.0f, 1.0f));

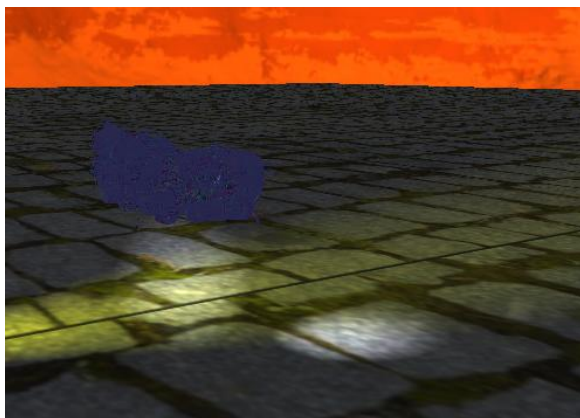
glm::vec3 dirFrenteLocal = glm::normalize(glm::vec3(1.0f, -1.5f, 0.0f));
glm::vec3 dirAtrasLocal = glm::normalize(glm::vec3(-1.0f, -1.5f, 0.0f));

glm::vec3 dirFrenteWorld = glm::normalize(glm::mat3(modelNaveRender) * dirFrenteLocal);
glm::vec3 dirAtrasWorld = glm::normalize(glm::mat3(modelNaveRender) * dirAtrasLocal);

if (mainWindow.getKeys()[GLFW_KEY_H])
{
    spotLights[3].SetFlash(naveSpotPosOff, dirFrenteWorld);
    spotLights[4].SetFlash(naveSpotPosTrasero, dirAtrasWorld);
}
else if (mainWindow.getKeys()[GLFW_KEY_J])
{
    spotLights[3].SetFlash(naveSpotPosFrontal, dirFrenteWorld);
    spotLights[4].SetFlash(naveSpotPosOff, dirAtrasWorld);
}
else
{
    spotLights[3].SetFlash(naveSpotPosOff, dirFrenteWorld);
    spotLights[4].SetFlash(naveSpotPosOff, dirAtrasWorld);
}

```

spotLights[3] y spotLights[4] representan los dos conos de luz de la nave. Cuando se presiona una tecla, uno de ellos se posiciona en el frente o atrás de la nave y el otro se manda fuera de escena con naveSpotPosOff, simulando encendido y apagado sin alterar la estructura del arreglo de luces.



2.- Crear o instanciar una pecera (traslúcida) con las normales hacia adentro y en la parte superior imagen de agua e instanciar dentro de la pecera al pez abisal con movimiento de teclado para subir y bajar en diagonal.

Se construyó una pecera con geometría propia para vidrio y agua. El vidrio se modeló con normales dirigidas hacia adentro y se renderizó con transparencia; además, se instanció un pez abisal dentro del volumen de la pecera usando una jerarquía basada en peceraBase y pezBaseNoScale. El pez puede desplazarse sobre el eje Z y moverse en diagonal hacia arriba o hacia abajo con control de teclado.

Bloque de código para captura de la pecera:

```
void CrearPecera()
{
    const GLfloat hx = peceraHX;
    const GLfloat hy = peceraHY;
    const GLfloat hz = peceraHZ;

    const GLfloat hxAgua = hx - 0.05f;
    const GLfloat hzAgua = hz - 0.05f;
    const GLfloat yAgua = hy - 0.08f;

    const GLfloat guL = 35.0f / 576.0f;
    const GLfloat guR = 548.0f / 576.0f;
    const GLfloat gvT = 1.0f - (60.0f / 576.0f);
    const GLfloat gvB = 1.0f - (496.0f / 576.0f);

    const GLfloat auL = 58.0f / 576.0f;
    const GLfloat auR = 519.0f / 576.0f;
    const GLfloat avT = 1.0f - (63.0f / 576.0f);
    const GLfloat avB = 1.0f - (478.0f / 576.0f);

    unsigned int vidrioIndices[] = {
        0, 1, 2, 0, 2, 3,
        4, 5, 6, 4, 6, 7,
        8, 9, 10, 8, 10, 11,
        12, 13, 14, 12, 14, 15
    };

    GLfloat vidrioVertices[] = {
        -hx, -hy, hz, guL, gvB, 0.0f, 0.0f, -1.0f,
        hx, -hy, hz, guR, gvB, 0.0f, 0.0f, -1.0f,
        hx, hy, hz, guR, gvT, 0.0f, 0.0f, -1.0f,
        -hx, hy, hz, guL, gvT, 0.0f, 0.0f, -1.0f,

        hx, -hy, -hz, guL, gvB, 0.0f, 0.0f, 1.0f,
        -hx, -hy, -hz, guR, gvB, 0.0f, 0.0f, 1.0f,
        -hx, hy, -hz, guR, gvT, 0.0f, 0.0f, 1.0f,
        hx, hy, -hz, guL, gvT, 0.0f, 0.0f, 1.0f,

        -hx, -hy, -hz, guL, gvB, 1.0f, 0.0f, 0.0f,
        -hx, -hy, hz, guR, gvB, 1.0f, 0.0f, 0.0f,
        -hx, hy, hz, guR, gvT, 1.0f, 0.0f, 0.0f,
        -hx, hy, -hz, guL, gvT, 1.0f, 0.0f, 0.0f,

        hx, -hy, hz, guL, gvB, -1.0f, 0.0f, 0.0f,
        hx, -hy, -hz, guR, gvB, -1.0f, 0.0f, 0.0f,
        hx, hy, -hz, guR, gvT, -1.0f, 0.0f, 0.0f,
        hx, hy, hz, guL, gvT, -1.0f, 0.0f, 0.0f
    };

    unsigned int aguaIndices[] = {
        0, 1, 2,
        0, 2, 3
    };
}
```

Bloque de código para captura del movimiento del pez:

```
// Movimiento del pez dentro de la pecera
avanza = mainWindow.getKeys()[GLFW_KEY_I];
retrocede = mainWindow.getKeys()[GLFW_KEY_K];
sube = mainWindow.getKeys()[GLFW_KEY_U];
baja = mainWindow.getKeys()[GLFW_KEY_O];

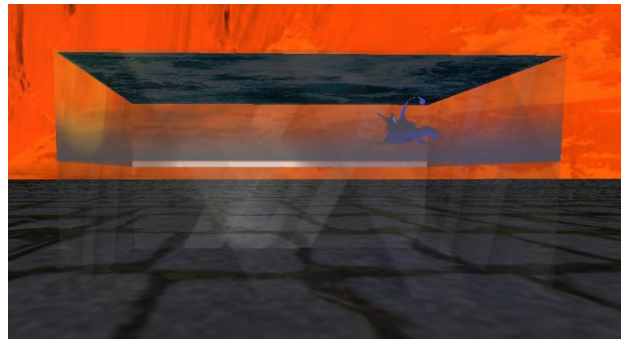
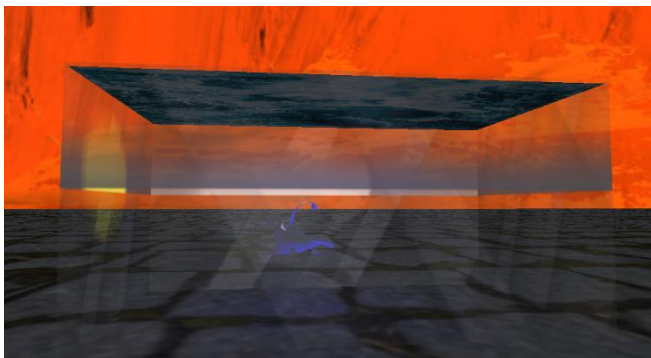
deltaZ = 0.0f;
deltaY = 0.0f;

if (sube)
{
    deltaY += velocidadPez * deltaTime;
    deltaZ -= velocidadPez * deltaTime;
}
else if (baja)
{
    deltaY -= velocidadPez * deltaTime;
    deltaZ -= velocidadPez * deltaTime;
}
else
{
    if (avanza)
        deltaZ -= velocidadPez * deltaTime;
    if (retrocede)
        deltaZ += velocidadPez * deltaTime;
}

pezPosLocal.z += deltaZ;
pezPosLocal.y += deltaY;

pezPosLocal.z = clampf(pezPosLocal.z, -peceraHZ + 0.9f, peceraHZ - 0.9f);
pezPosLocal.y = clampf(pezPosLocal.y, -peceraHY + 0.8f, peceraHY - 1.0f);
```

la pecera se compone de una malla translúcida para el vidrio y un plano superior con textura de agua. El pez hereda primero la transformación de la pecera y luego su propia traslación y rotación, lo que permite mantenerlo contenido dentro del acuario. El uso de clampf evita que el pez atraviase las paredes.



3.- Agregar una luz de tipo puntual de color azul ligada al bulbo del pez que puedan prender y apagar de forma independiente con teclado tanto la luz de la lámpara de su práctica 7 como esta luz (la luz de la lámpara debe de ser puntual, si la crearon spotlight en su reporte 7 tienen que cambiarla a luz puntual), de tal forma que se pueda ver: las 2 luces apagadas, las 2 luces prendidas, una luz prendida y una luz apagada y viceversa.

Se implementaron dos PointLight independientes: una blanca para la lámpara y una azul para el bulbo del pez. Ambas pueden activarse y desactivarse con teclado de forma independiente, lo que permite demostrar las cuatro combinaciones requeridas: ambas apagadas, ambas encendidas, solo lámpara encendida y solo pez encendido. OpenGL y GLSL permiten manejar varias luces y habilitarlas individualmente en cada actualización del shader.

Bloque de código para captura de creación de luces:

```
// Luz puntual de la lámpara
pointLights[1] = PointLight(1.0f, 1.0f, 1.0f,
    0.2f, 3.0f,
    posLampara.x, posLampara.y + 1.0f, posLampara.z,
    1.0f, 0.7f, 1.8f);
pointLightCount++;

// Luz puntual azul ligada al bulbo del pez
pointLights[2] = PointLight(0.0f, 0.0f, 1.0f,
    0.1f, 3.0f,
    posPecera.x, posPecera.y, posPecera.z,
    1.0f, 0.7f, 1.8f);
pointLightCount++;
```

Bloque de código para captura de teclado y activación:

```
// Encendido independiente de la lámpara
if (mainWindow.getKeys()[GLFW_KEY_1])
{
    if (!teclaLamparaPresionada)
    {
        lamparaEncendida = !lamparaEncendida;
        teclaLamparaPresionada = true;
    }
}
else
{
    teclaLamparaPresionada = false;
}

// Encendido independiente de la luz azul del pez
if (mainWindow.getKeys()[GLFW_KEY_2])
{
    if (!teclaBulboPezPresionada)
    {
        luzBulboPezEncendida = !luzBulboPezEncendida;
        teclaBulboPezPresionada = true;
    }
}
else
{
    teclaBulboPezPresionada = false;
}
```

La lámpara se controla con la tecla 1 y la luz azul del pez con la tecla 2. La posición de la lámpara se actualiza desde `modelLampara`, mientras que la del pez sigue el `bulbo` con `bulboWorld`. Después, sólo se envían al shader las luces activas mediante el arreglo `pointLightsActivas`

Ambos apagados



Lámpara encendida, (con tecla 1)



Pez encendido lampara apagada (con tecla 2)

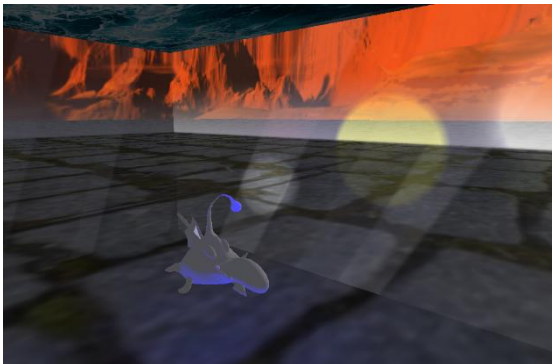


Ambos encendidos



4. *Agregar un spotlight (que no sea luz de color blanco ni azul) que parta del bulbo del pez y que ilumine en cualquier dirección dentro de la pecera al presionar teclas asignadas al eje x, eje y y eje z.*

Se añadió un spotlight amarillo, distinto de blanco y azul, que sale del bulbo del pez y puede orientarse dentro de la pecera mediante control angular. La dirección del cono se calcula con ángulos yaw y pitch, y después se transforma con la matriz del pez para que herede correctamente su orientación en el mundo. Esta técnica es equivalente a rotar un vector local mediante la transformación del modelo para convertirlo a coordenadas globales.



Bloque de código::

```
// Spotlight del pez
spotLights[5] = SpotLight(1.0f, 1.0f, 0.0f,
    0.0f, 2.0f,
    posPecera.x, posPecera.y, posPecera.z,
    0.0f, 0.0f, 1.0f,
    1.0f, 0.0f, 0.0f,
    18.0f);
spotLightCount++;
```

```
// Dirección del spotlight del pez
if (mainWindow.getKeys()[GLFW_KEY_LEFT]) spotPezYaw -= velocidadSpotPez * deltaTime;
if (mainWindow.getKeys()[GLFW_KEY_RIGHT]) spotPezYaw += velocidadSpotPez * deltaTime;
if (mainWindow.getKeys()[GLFW_KEY_UP]) spotPezPitch += velocidadSpotPez * deltaTime;
if (mainWindow.getKeys()[GLFW_KEY_DOWN]) spotPezPitch -= velocidadSpotPez * deltaTime;

spotPezPitch = clampf(spotPezPitch, -80.0f, 80.0f);
```

El spotlight se inicializa con color amarillo y después su dirección se actualiza con las flechas del teclado. LEFT y RIGHT modifican el giro horizontal, mientras que UP y DOWN modifican la inclinación vertical. La transformación con glm::mat3(pezBaseNoScale) hace que la luz parta con la misma orientación general del pez

Controles de teclado utilizados en la práctica

<i>Tecla</i>	<i>Función</i>
1	Enciende o apaga la luz puntual de la lámpara.
2	Enciende o apaga la luz puntual azul del bulbo del pez.
F	Cambia el color del faro del carro.
H	Activa el spotlight trasero de la nave al moverse en dirección correspondiente.
J	Activa el spotlight frontal de la nave al moverse en dirección correspondiente.
I	Mueve al pez hacia adelante dentro de la pecera sobre el eje Z.
K	Mueve al pez hacia atrás dentro de la pecera sobre el eje Z.
U	Mueve al pez en diagonal hacia arriba.
O	Mueve al pez en diagonal hacia abajo.
←	Gira el spotlight del pez hacia la izquierda sobre el eje horizontal.
→	Gira el spotlight del pez hacia la derecha sobre el eje horizontal.
↑	Inclina el spotlight del pez hacia arriba.
↓	Inclina el spotlight del pez hacia abajo.

2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

Durante el desarrollo de la práctica se presentaron varios problemas técnicos:

- Uno de los problemas principales fue que el suelo no recibía la iluminación del spotlight, aunque la nave y el carro sí parecían tener la luz activa. Se detectó que la causa era la normal del piso apuntando hacia abajo en floorVertices (0.0f, -1.0f, 0.0f), y se corrigió cambiándola a 0.0f, 1.0f, 0.0f para que la iluminación difusa pudiera calcularse correctamente sobre el plano.
- También hubo un problema con la dirección y posición de los spotlights del carro y de la nave, porque el haz no caía correctamente sobre el suelo o quedaba mal orientado respecto al modelo. Esto se resolvió recalculando las posiciones y direcciones de las luces a partir de las matrices de transformación de cada objeto, usando la jerarquía del modelo para que el spotlight siguiera el movimiento real del carro y de la nave.
- Otro problema fue la confusión en la lógica de las teclas J y H, ya que al principio no coincidía qué tecla movía la nave y cuál luz debía encenderse en cada caso. La solución fue ajustar la condición de activación para que solo se encendiera el reflector correspondiente a la dirección real del movimiento, dejando apagado el reflector contrario.
- También se presentó el problema de que apagar una luz solo cambiando el color no siempre daba el efecto esperado, porque el spotlight seguía existiendo en la escena. Para resolverlo, se usó una lógica de “apagado” moviendo temporalmente la luz fuera de la escena con una posición off, de modo que el reflector inactivo dejara de afectar visualmente al suelo y a los objetos.
- Alineación del pez y su spotlight: al rotar el pez 180 grados, el spotlight del bulbo no iniciaba apuntando en la misma dirección. Se resolvió calculando la dirección del spotlight con pezBaseNoScale en lugar de usar sólo la transformación de la pecera.
- Movimiento del pez: en un principio el movimiento vertical dependía de presionar varias teclas al mismo tiempo. Se simplificó la lógica haciendo que U y O produjeran directamente desplazamiento diagonal.
- Iluminación del vidrio por fuera: al renderizar ambas caras del vidrio y además invertir normales en el shader, la iluminación se veía incorrectamente en el exterior. Se resolvió ajustando el manejo de normales en el shader y manteniendo una configuración de render que permitiera iluminar correctamente las paredes internas.

- Finalmente, se tuvo que revisar que el problema no estuviera en el shader o en el manejo de normales, ya que en OpenGL la iluminación depende directamente de la normal de la superficie y de que esté en el mismo sistema de referencia que la luz. Después de revisar eso, se concluyó que el error principal estaba en la normal del piso y en la calibración de los reflectores, no en la estructura general de la escena.

3.- Conclusión:

La práctica permitió integrar distintos tipos de iluminación en una misma escena 3D, combinando luz direccional, luces puntuales y spotlights asociados a objetos con movimiento. El aspecto más importante del desarrollo fue comprender que una luz ligada a un modelo no sólo requiere una posición, sino también una dirección calculada a partir de la transformación del objeto, especialmente cuando se usan jerarquías de matrices para pez, bulbo, nave y lámpara.

También se reforzó el manejo de transparencia, normales y control interactivo por teclado. La pecera translúcida, el pez con movimiento diagonal y las luces independientes evidencian que una escena con varios elementos dinámicos requiere coordinar correctamente geometría, shaders, materiales y actualización de estados por fotograma.

Bibliografía

- Sellers, G., Wright, R. S., y Haemel, N. (2015). OpenGL superbible: Comprehensive tutorial and reference (7th ed.). Addison-Wesley.
- Shreiner, D., Sellers, G., Kessenich, J., y Licea-Kane, B. (2013). OpenGL programming guide: The official guide to learning OpenGL (8th ed.). Addison-Wesley.
- The Khronos Group. (2024). OpenGL reference pages. <https://registry.khronos.org/OpenGL-Refpages/gl4>
- Madhav, S. (2020). Game programming algorithms and techniques: A platform-agnostic approach. Pearson Education.